

数 学 建 模

题 目 基于土壤湿度预测的智能灌溉系统优化设计

学 院： 理 学 院
专 业： 信息与计算科学

姓 名：	<u>杨进</u>	学 号：	<u>1131230208</u>
姓 名：	<u>吴雨润</u>	学 号：	<u>1131230201</u>
姓 名：	<u>林鑫</u>	学 号：	<u>1043220122</u>
姓 名：	<u>赵文娇</u>	学 号：	<u>1131230204</u>

二〇二五 年 七月十一日

基于土壤湿度预测的智能灌溉系统优化设计

摘要

针对农业灌溉水资源浪费与效率低问题，围绕基于土壤湿度预测的智能灌溉系统优化设计展开研究。

对于问题一，土壤类型固定为砂土，以初始湿度 40%为基础，构建土壤湿度指数衰减方程 $\theta_t = \theta_0 e^{-kt}$ ，数据拟合得到日照强度与衰减系数的函数关系： $k = aR + b$ ，利用无锡历史数据训练模型，预测出 7 月 12 日至 14 日 48 小时湿度变化曲线，为湿度动态预测提供依据。

对于问题二，引入温度、风速、降水量等气象参数，搭建 BAS-BP 神经网络预测模型，将多源数据标准化后作为输入，土壤湿度为输出，经训练后，在连续晴天/雨天场景验证，得到均方误差 RMSE 和平均绝对误差 MAE，并以此为指标来衡量预测误差，提升湿度预测精度与气象数据耦合性。

对于问题三，聚焦玉米抽穗期，设定最适湿度区间[65%,75%]，以灌溉次数最少为目标函数，推导触发灌溉的湿度阈值（低于 65%触发），通过修正后的 Penman-Monteith 公式计算作物蒸腾量，并构建土壤湿度变化方程，通过每日更新，实现灌溉决策优化。系列模型协同，助力解决智能灌溉“何时灌、灌多少”核心问题，为节水高效农业提供数学支撑。

关键词：智能灌溉；土壤湿度预测；BP 神经网络；灌溉决策；指数衰减模型

一、问题重述

1.1 问题背景

土壤湿度是全球水文循环的重要组成部分，影响水圈、大气圈、生物圈之间的能量和物质传输。准确的土壤湿度信息在水旱灾害监测、季节性水文预报和农业管理等方面具有重要的应用价值^{[1][2][3]}。全球淡水资源短缺背景下，农业灌溉用水占比高达 70%，但传统灌溉方式效率不足 50%。我国华北平原等地区因过度灌溉导致地下水位年均下降 1-3 米，而智慧农业设备普及率不足 15%。通过数学建模优化自动灌溉系统，可实现：①动态匹配作物需水量 ②减少 30%-60%水资源浪费 ③降低农田盐碱化风险。本研究要求建立多参数耦合的智能决策模型，解决“何时灌”“灌多少”的核心问题。亟需解决的关键问题：土壤湿度动态预测精度不足（现有模型误差 $\pm 20\%$ ）、气象数据与灌溉决策的实时耦合困难、不同作物生长阶段的差异化需水规律、系统能耗与节水效益的平衡优化。

1.2 问题概述

在全球淡水资源短缺、无锡农业灌溉面临效率低与智慧化不足的背景下，聚焦基于土壤湿度预测的智能灌溉系统优化设计。

要求针对题目背景和附件数据解决下面三个问题：

问题一：针对无锡土壤（砂土/黏土），构建单点土壤湿度指数衰减模型，关联日照强度与衰减系数，利用历史数据预测并输出无锡市未来 48 小时湿度变化曲线。

问题二：引入气象参数构建预测系统，整合无锡气象数据（温度、风速、降水量等），建立 BAS-BP 神经网络预测模型，验证连续晴/雨天场景下的预测误差。

问题三：围绕玉米抽穗期，设定玉米抽穗期最适湿度区间 $[65\%, 75\%]$ ，以灌溉次数最少为目标，设计最优灌溉决策算法，明确触发灌溉湿度阈值与水量计算公式，实现精准节水灌溉。

二、问题分析

2.1 整体分析

本研究采用“基础模型构建→多参数扩展→决策优化”的三阶段递进式研究框架，系统解决智能灌溉中的核心问题。

2.2 问题一

要建立单点土壤湿度衰减模型，需考虑土壤类型和日照强度对土壤湿度衰减的影响。不同土壤类型的物理性质不同，其水分保持和蒸发特性也不同。日照强度会影响土壤表面的温度和水分蒸发速率，从而影响土壤湿度的衰减。通过收集无锡市若干年的相关数据，利用指数衰减方程拟合数据，分析日照强度与衰减系数的关系，进而预测未来 48 小时的湿度变化曲线。

2.3 问题二

引入气象参数构建预测系统，是为了提高土壤湿度预测的精度。温度、风速、降水量等气象因素都会对土壤湿度产生影响。BP 神经网络具有较强的非线性映射能力，能够处理多输入多输出的复杂关系。但传统 BP 神经网络存在参数初始化敏感和易陷入局部极小值问题。引入 BAS 算法进行初始权重搜索，有助于找到较优初始点，增强网络学习能力。此外，数据标准化是神经网络训练的关键步骤，能够加速收敛，提高数值稳

定性。通过将气象数据作为输入，土壤湿度作为输出，训练 BAS-BP 神经网络模型，然后在连续晴天和雨天场景下验证模型的预测误差。

2.4 问题三

本问题是一个典型的农业水资源管理优化问题，目标是在确保作物正常生长的基础上，实现水资源的有效利用。

1. 蒸散发量的准确估算：

作物蒸散发是土壤水分消耗的主要途径，受气象条件、作物类型、生长阶段等多种因素影响。传统的经验公式往往精度不足，而物理模型(如 Penman-Monteith)能更好地反映实际情况。

2. 土壤水分动态平衡模拟：

土壤湿度是一个动态变化的量，受降雨、蒸散发和灌溉的影响。需要建立一个水文平衡模型来模拟土壤湿度的日变化，作为灌溉决策的依据。

3. 作物生长阶段影响：

不同生长阶段的作物对水分的需求不同，通过引入作物系数(K_c)可以反映这种差异，使模型更具作物特异性。

4. 灌溉阈值设定：

如何确定土壤湿度的最优范围(上限和下限)以及触发灌溉的阈值，是保证作物健康和节水的关键。当土壤湿度低于下限时，需要启动灌溉，并计算灌溉量以将其恢复到理想范围。

2.5 数据需求

本论文所有数据来源于 ECMWF 网站，该网站提供全球气象数据：

1、土壤数据：

- 1) 无锡地区土壤类型分布
- 2) 历史土壤湿度观测数据（附件 1）

2、气象数据：

- 1) 日照强度（附件 2）
- 2) 气温、土壤湿度(附件 3)
- 3) 风速、降水量（附件 3）

三、模型假设及符号说明

3.1 问题一

1. 模型假设

- 1) 土壤特性均匀稳定，不考虑土壤内部结构差异对水分运动的影响。
- 2) 土壤类型固定为砂土，其水分蒸发过程可视为指数衰减过程。
- 3) 降水、风速、温度等对土壤湿度的影响在本问题中忽略不计。
- 4) 土壤湿度变化主要受日照强度影响, 忽略作物根系吸水的干扰。
- 5) 土壤湿度数据与辐射强度数据记录均为小时尺度, 时间统一转为北京时间。
- 6) 历史相同日期的短波辐射强度与土壤湿度可以代表未来对应日期的特征。

2. 符号说明

符号	含义	单位
θ_t	t 时刻土壤湿度	%
θ_0	初始土壤湿度, 定义为 40%	%
k	基础衰减系数	-
R	日照强度	W/m^2
t	时间	h

3.2 问题二

1、模型假设

- 1) 土壤湿度与风速、降水量、温度、下行短波辐射强度等气象因素存在一定的关联性, 且可通过历史数据进行建模预测。
- 2) 假设作物生长过程中除土壤湿度和气象条件外, 其他因素 (如病虫害、施肥等) 对需水量的影响可忽略不计。
- 3) 气象数据采集准确且传输无延迟, 在预测时间段内具有一定的稳定性和可预测性, 不考虑突发极端气象事件。
- 4) 时序滞后特征对土壤湿度的预测具有显著贡献, 适当引入后项能提高模型表现。
- 5) BAS 算法可有效优化 BP 神经网络的初始权重, 避免陷入局部最优, 提升模型收敛速度和精度。
- 6) 模型训练采用标准化处理。

3.3 问题三

1、模型假设

- 1) 土壤在整个灌溉区域内是均匀的, 即土壤的质地、孔隙度、容重等物理性质在空间上没有显著差异。
- 2) 简化土壤水分的运动过程, 不考虑水分的侧向流动、深层渗漏等复杂的物理过程。
- 3) 灌溉区域内只种植单一作物 (玉米), 并且该作物处于抽穗期, 整个生长阶段对土壤湿度的需求相对稳定。
- 4) 所有玉米植株对水分的吸收、利用和响应是一致的, 不考虑作物根系分布差异、病虫害等因素对作物水分需求的影响。
- 5) 在模拟的时间范围内, 气象条件相对稳定, 不考虑突发的暴雨、大风等极端天气事件对土壤湿度的影响。
- 6) 灌溉水全部被土壤吸收, 没有任何损失, 灌溉效率为 100%。
- 7) 灌溉设备工作效率稳定, 能够将水均匀施加到土壤中, 不存在灌溉不均匀的情况。
- 8) 输入的土壤湿度预测数据准确可靠, 能够真实反映未来一段时间内土壤湿度的变化情况。
- 9) 土壤湿度预测数据中的每个时间步是相互独立的, 不考虑相邻时间步之间的土壤湿度的相关性和连续性。

2、符号说明

符号	含义	单位
D	温度-饱和曲线在 T 处的曲线	$kPa/^\circ C$
R_n	净辐射	$MJ/(m^2 \cdot d)$
G	土壤热通量	$MJ/(m^2 \cdot d)$

γ	湿度常数	$kPa/^{\circ}C$
T	日平均气温	$^{\circ}C$
U_2	2 米处高度风速	m/s
e_s	饱和水汽压	kPa
e_a	实际水汽压	kPa

四、问题一模型的建立与求解

4.1 模型构建

1、单点土壤湿度衰减模型：

首先假设土壤湿度呈指数下降：

$$\theta_t = \theta_0 e^{-kt}$$

再考虑衰减系数与日照强度呈线性关系：

$$k = aR + b$$

得出总体指数衰减函数：

$$\theta_t = \theta_0 e^{-(aR+b)t}$$

2、参数拟合模型：

对每一天提取白天平均短波辐射 R_i 和实际湿度序列 $\theta_i(t)$ ，采用最小二乘法拟合得到日衰减系数 k_i ，对所有 (R_i, k_i) 数据点线性拟合，得到具体公式。

3、未来预测模型：

利用 2021-2024 年同日（7 月 12 日、13 日）的辐射数据估计未来辐射强度 R_{future} ，代入拟合模型得到 k_{future} ，再代入指数模型预测湿度。

4.2 模型求解

模型求解过程分为如下步骤：

1. 数据预处理：

- 1) 收集无锡市 2025 年 6 月 30 日之前若干年的土壤湿度（附件 1）、日照强度（附件 2）和土壤类型数据，将数据统一单位并转换为北京时间。
- 2) 处理下行辐射单位 (W/m^2) 与土壤湿度单位 $(m^3/m^3 \rightarrow \%)$ 。
- 3) 对每一天提取辐射强度与湿度序列。

2. 衰减系数拟合：

- 1) 对每日土壤湿度数据 θ_t 分别取对数，利用 $\ln \theta_t = \ln \theta_0 - kt$ 线性拟合求解 k_i 。
- 2) 对应计算该日平均辐射 R_i 。

3. 构建 $k - R$ 回归模型：

使用 sklearn 的 LinearRegression 对 (R_i, k_i) 进行元线性回归。

4. 预测未来湿度：

- 1) 估计 2025 年 7 月 12 日至 13 日白天平均辐射强度 R 。
- 2) 代入回归方程预测 k ，再用指数模型计算湿度衰减曲线。

4.3 模型结果

1. 拟合结果：

通过分析历史多日数据，拟合得到如下关系：

$$k=0.00000310R+0.00028681 \text{ (图 4-1, 图 4-2)}$$

说明辐射强度越大，土壤湿度下降越快，验证了日照对土壤蒸发的显著驱动作用。

2. 预测结果:

- 1) 得出图像展示典型 k 值的衰减模式（图 4-3）。
- 2) 对 2025 年 7 月 12 日至 13 日进行湿度预测，得出 48 小时的衰减曲线（图 4-4）。
- 3) 与 2021-2024 年历史同期平均湿度曲线对比， 验证预测合理性（图 4-4）。

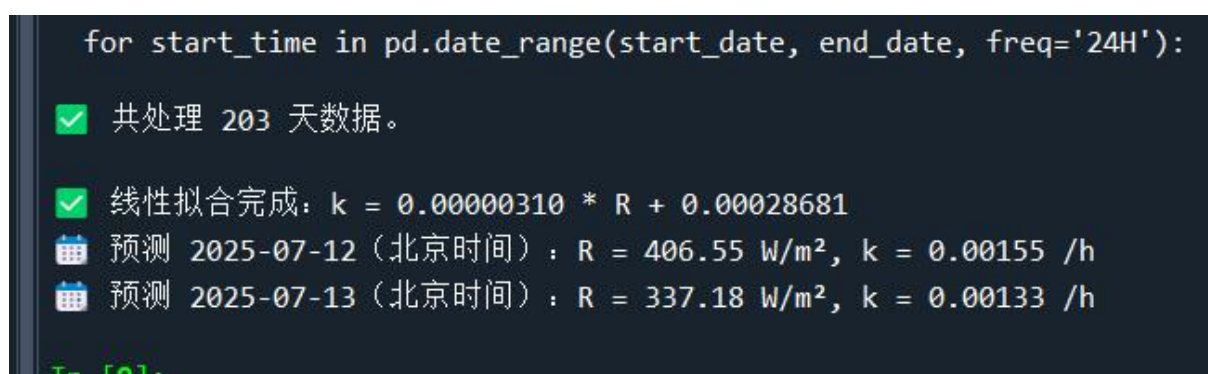


图 4-1 7 月 12 日至 14 日的日照强度和衰减系数

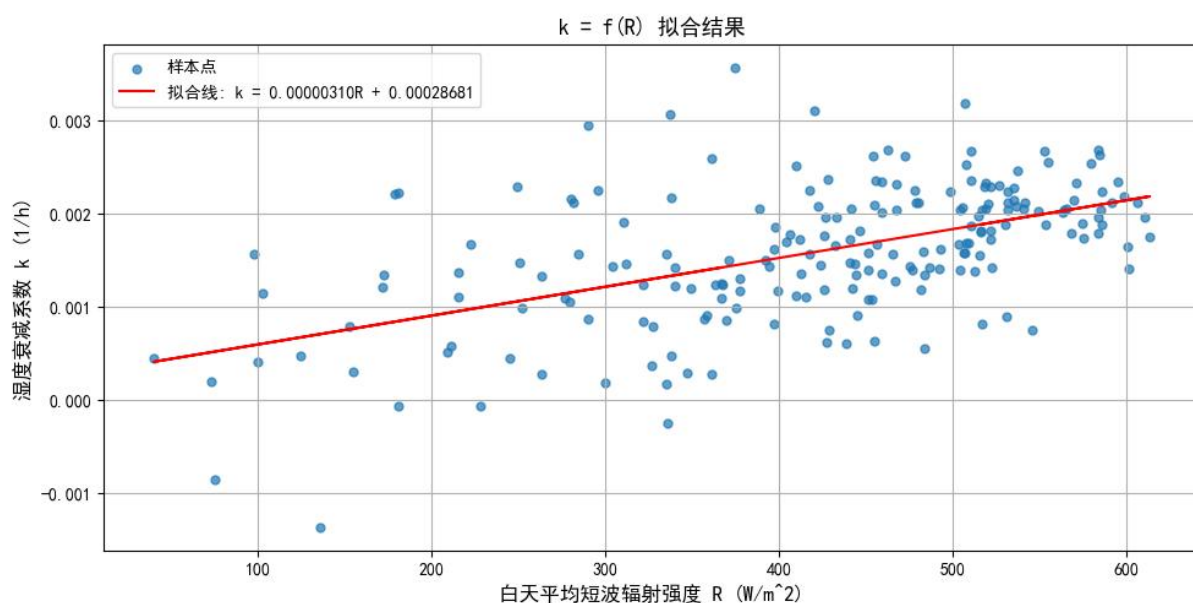


图 4-2 衰减系数与日照强度的关系

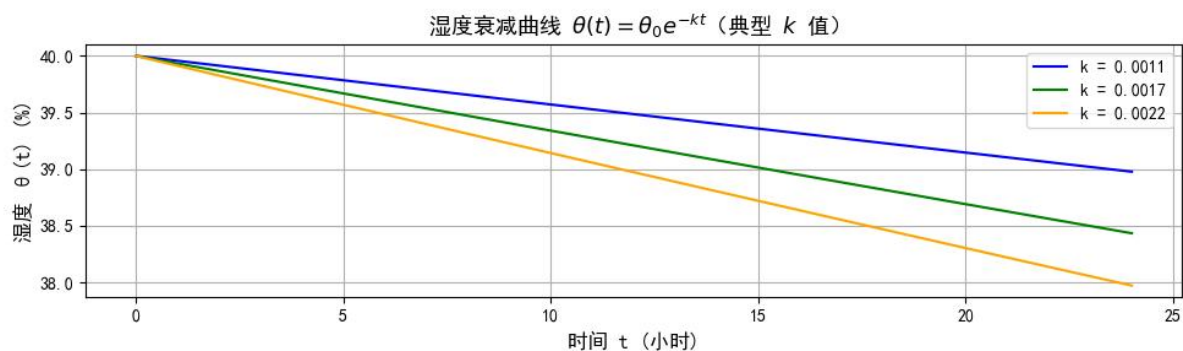


图 4-3 不同 k 值下土壤湿度衰减曲线

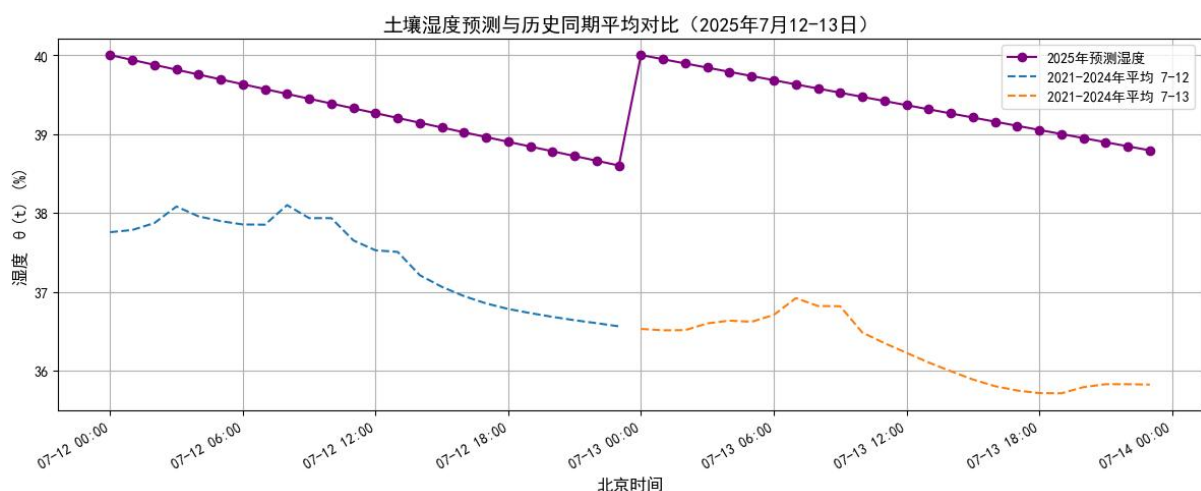


图 4-4 未来 48 小时土壤湿度预测曲线

五、问题二模型的建立与求解

5.1 模型构建

BP 神经网络由输入层、隐藏层和输出层组成。输入层的节点为温度 T 、风速 V 、降水量等气象参数，输出层的节点为土壤湿度。

网络的训练过程包括前向传播和反向传播。前向传播时，输入数据通过隐藏层的神经元进行非线性变换，网络的权值和偏移量保持不变，每层神经元的状态只影响下一层神经元的状态。当输出层达不到预期的输出时，可切换到误差信号的反向传播。反向传播时，根据输出结果与实际值的误差，调整网络的权值和阈值，使误差平方和最小^[4]。

1. 数据预处理模型

定义统一的数据读取函数，批量加载各气象变量和土壤湿度的 Excel 数据文件，去除单位符号，转换为数值型，并将时间列转为 pandas 的 Datetime 格式，合并各个变量数据，选取 6-7 月数据段，构建完整的特征矩阵与目标向量。

2. 特征与标签

选取风速、降水量、温度、下行短波辐射四个气象变量作为输入特征 X ，土壤湿度百分比作为预测目标 Y 。

3. 标准化处理

对输入特征和标签分别做均值方差标准化，使数据符合神经网络训练需求，避免量纲差异影响模型收敛。

4. BAS-BP 神经网络预测模型

基于 BAS 算法的优势，将其与 BP 神经网络结合，构成了对 BP 神经网络进行优化的 BAS-BP 模型。该模型预测流程如图 5-1^[5]。

5. 连续天气段误差分析

基于降水量判断晴天（降水=0）和雨天（降水>0），划分连续天气段，筛选长度超过阈值（12 小时）的时间段，分别计算这些连续段内模型预测的 RMSE 和 MAE。

6. 特征工程

为了捕捉土壤湿度的时间依赖性，此模型引入了时间滞后特征。对当前及滞后的 1 小时、3 小时、6 小时的风速、降水量、温度、下行辐射强度和土壤湿度作为模型的输入特征。经过滞后处理后对数据集进行了 NaN 值移除，确保数据的完整性。最终输入特征数量为 19 个（4 个当前气象特征+5 个滞后特征*3 个滞后步长）。

5.2 模型求解

1、BP 神经网络结构

本 BP 神经网络采用多层感知机结构，包含一个输入层、三个隐藏层和一个输出层。

- 1) 输入层神经元数： 19 （根据处理后的特征数量）
- 2) 隐藏层神经元数： (128,64,32)
- 3) 输出层神经元数： 1 （对应土壤湿度预测值）
- 4) 激活函数：隐藏层采用 ReLU 作为激活函数，输出层则不使用激活函数，以适应连续值预测任务。

5) 优化器：采用 Adam 优化器进行梯度下降，通过动态调整学习率来加速模型训练。

6) 训练参数：

- ① 学习率： 0.001
- ② 批大小： 64
- ③ 训练轮次： 3000
- ④ 验证集比例： 0.1
- ⑤ 早停机制： 3000 （实际操作中近似无早停，允许模型充分训练）

2、BAS 算法优化初始参数

传统的 BP 神经网络训练容易陷入局部最优，且对初始权值和偏置敏感。为解决此问题，本研究在 BP 神经网络训练前，利用 BAS 算法对网络的初始权重和偏置进行全局优化。BAS 算法通过模拟蝗虫的觅食行为，在参数空间中进行搜索，以找到一个能够使 BP 网络在训练初期达到更低 MSE（均方误差）的初始参数集。

BAS 训练参数：

- ① 最大迭代次数： 500
- ② 初始步长： 0.5
- ③ 步长衰减系数： 0.98

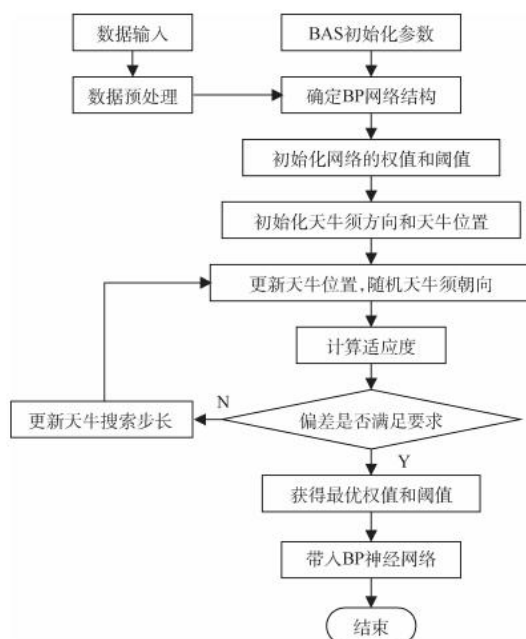


图 5-1 BAS-BP 模型的预测流程

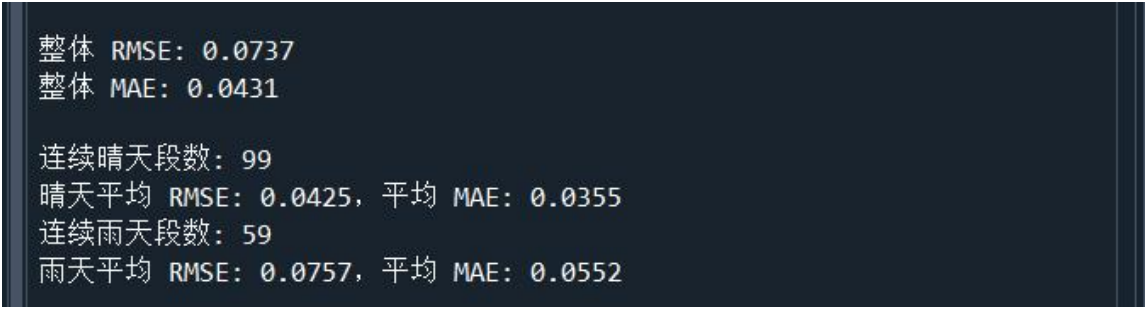


图 5-2 预测误差 RMSE 和 MAE

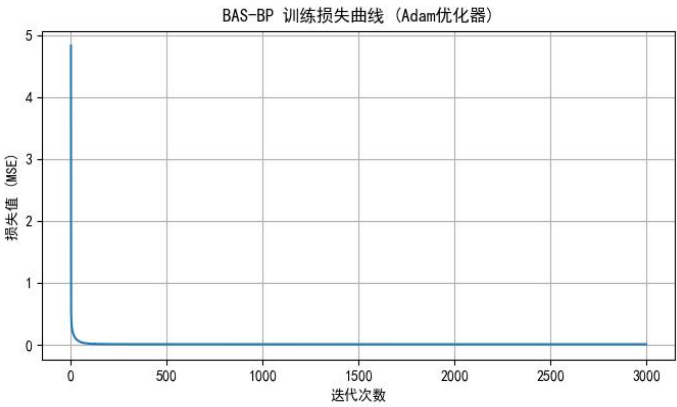


图 5-3 训练损失曲线

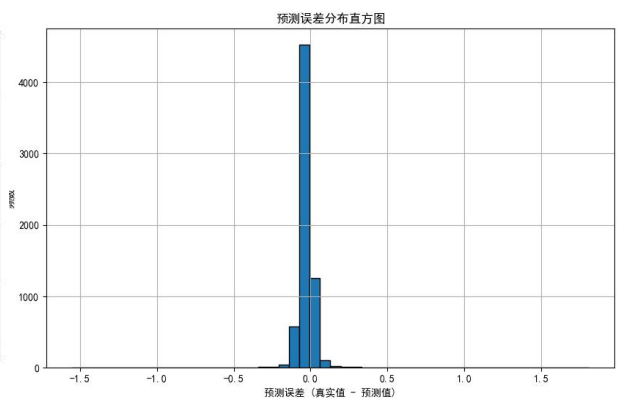


图 5-4 预测误差分布直方图

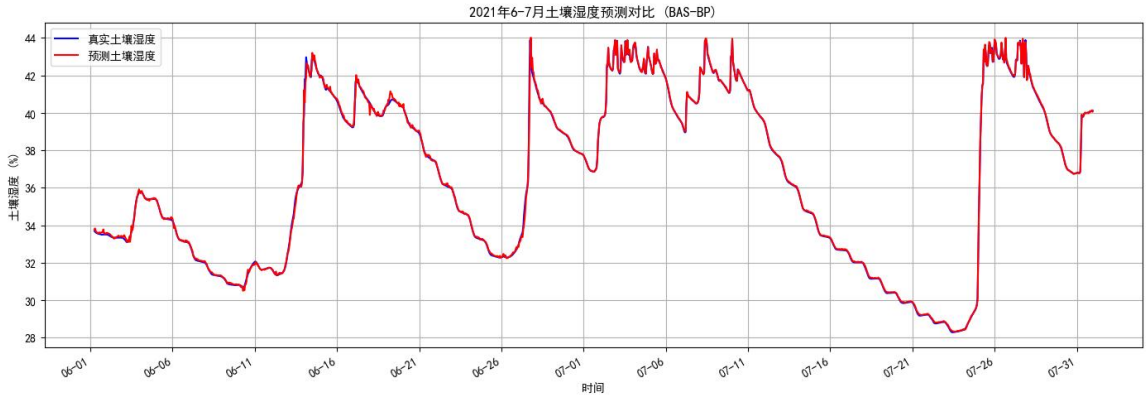


图 5-5 2021 年 6-7 月土壤湿度预测对比

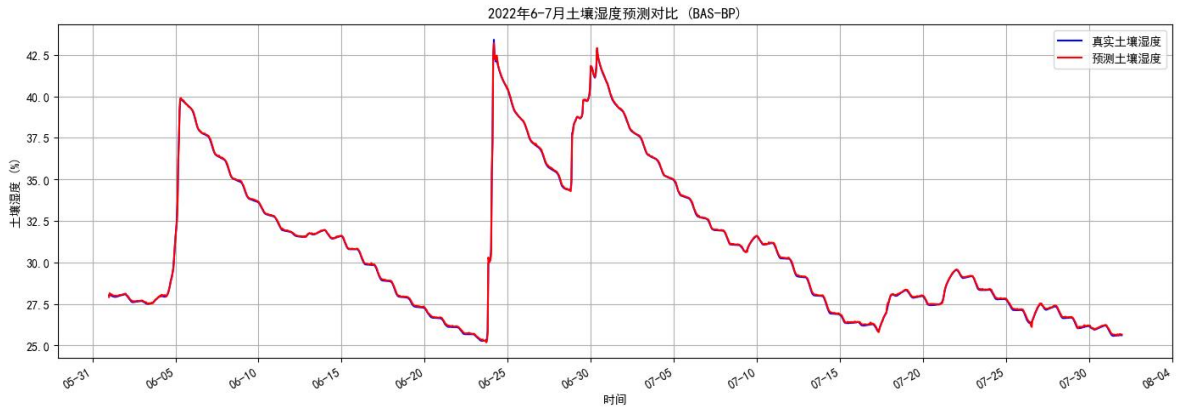


图 5-6 2022 年 6-7 月土壤湿度预测对比

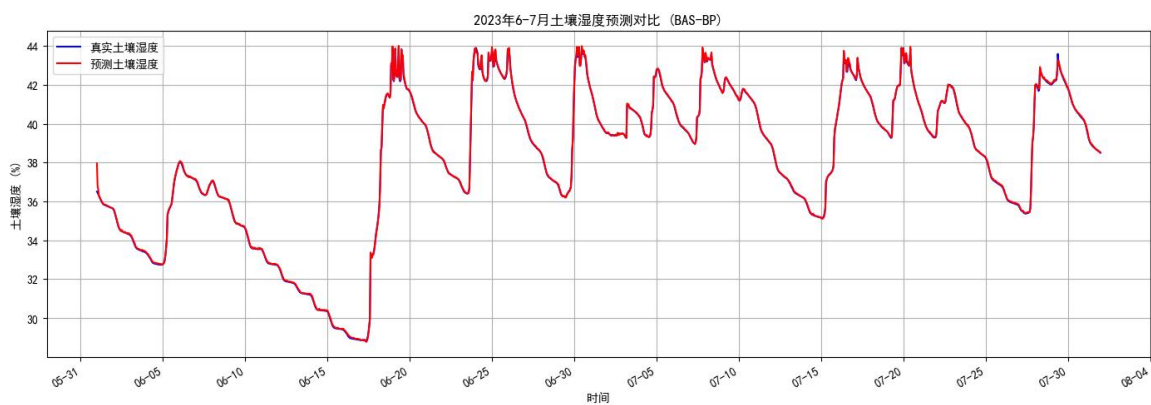


图 5-7 2023 年 6-7 月土壤湿度预测对比

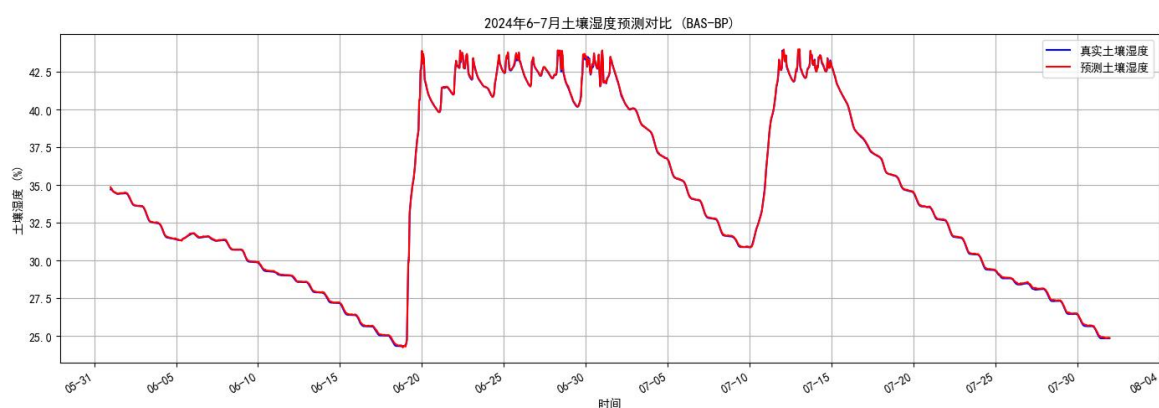


图 5-8 2024 年 6-7 月土壤湿度预测对比

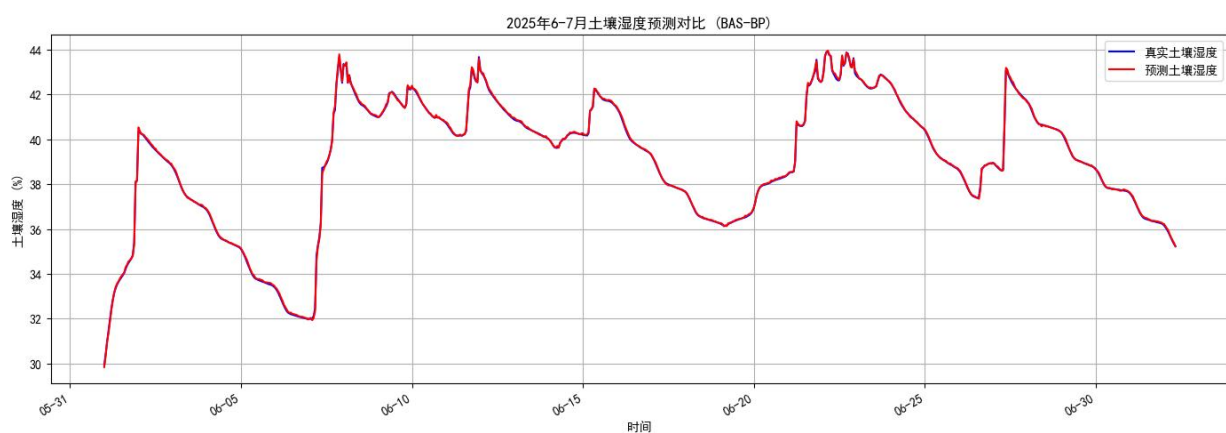


图 5-9 2025 年 6-7 月土壤湿度预测对比

5.3 模型结果

1. 整体预测性能：

模型在 6-7 月时间段的整体 RMSE 和 MAE 表明，所建立的 BAS-BP 神经网络能够较好地拟合土壤湿度与气象因素的非线性关系，实现有效预测。

2. 天气分段误差分析：

通过连续晴天和雨天时间段误差分析发现，模型在不同天气条件下的预测误差存在差异，通常雨天段的预测误差较晴天段更大，表明模型在降水等突变天气条件下的预测性能仍有提升空间。

六、问题三模型的建立与求解

6.1 模型构建

本模型采用基于物理过程的土壤水文平衡模型与 Penman-Monteith 公式相结合的方法，对玉米田的土壤湿度进行预测，并在此基础上做出灌溉决策。

1. 作物耗水量预测模型

采用经联合国粮农组织修正后的 FAO-56 Penman-Monteith 公式计算作物蒸发量^[6]：

$$ET_0 = \frac{0.408D(R_n - G) + \gamma \frac{900}{T + 273} U_2(e_s - e_a)}{D + \gamma(1 + 0.34U_2)}$$

再利用作物系数 K_c 对 ET_0 进行修正，得到作物实际蒸散发量 ET_c ：

$$ET_c = K_c \times ET_0$$

其中 K_c 根据玉米的生长阶段(初始阶段、中期阶段(抽穗期)、晚期阶段)动态取值，本模型采用抽穗期作物系数。

2. 土壤水分平衡模型

每日土壤湿度变化通过水文平衡方程进行模拟：

$$SW_t = SW_{t-1} + P_{eff} + I - ET_c - DP$$

其中 P_{eff} 为日有效降雨量， mm ；本题中用日平均降雨量代替， I 为灌溉水量， mm ； DP 为土壤深层渗透量， mm 。

3. 灌溉决策模型

灌溉决策基于预测的土壤湿度，如果预测的土壤湿度低于设定的最优湿度下限，并且当前日期在设定的灌溉阈值内，则触发灌溉。

当需要灌溉时，灌溉量计算为使土壤湿度达到最优湿度上限所需的水量：

$$\text{所需水分百分比} = \text{最优湿度上限} - \text{预测的土壤湿度}$$

$$\text{灌溉水量} = \text{所需水百分比} \times \text{田间面积} \times \text{作物根系深度}$$

灌溉后，将土壤湿度立即提升至最优湿度上限，作为次日预测的起始点；如果降雨或灌溉导致土壤湿度超过田间持水量，则将超出部分视为深层渗漏，并修正土壤湿度为田间持水量。

6.2 模型求解

模型的求解过程是一个每日迭代的模拟过程：

1. 数据加载与预处理。

2. 初始化：

设置初始土壤湿度，优先使用模拟开始日期的实际土壤湿度数据，否则采用默认值。

3. 每日循环模拟：

- 1) 模型对模拟日期范围内的每一天进行迭代。
- 2) 获取当日气象数据:
从合并后的 **DataFrame** 中提取当前日期的气温、风速、降雨量、辐射和露点温度。
- 3) 计算实际蒸散发量 ET_c 。
- 4) 模拟土壤湿度变化:
根据昨日预测结束的土壤湿度, 当日降雨量和当日 ET_c 量, 计算当日土壤湿度预测值。如果预测湿度超过田间持水量, 则将超出部分视为深层渗漏, 并将湿度限制在田间持水量。
- 5) 灌溉决策:
判断当前预测土壤湿度是否低于下限阈值。如果低于且在灌溉窗口期内, 则计算所需灌溉水量, 并将土壤湿度提升至上限。

6.3 模型结果

```

--- 基于气象数据 (包含露点温度)、Penman-Monteith 模型和作物阶段Kc的模拟灌溉决策 ---
模拟灌溉日期范围: 从 2025-06-07 到 2025-06-28

--- 详细灌溉事件列表 ---
日期: 2025-06-07 | 灌溉水量: 430.84 m³
日期: 2025-06-11 | 灌溉水量: 103.24 m³
日期: 2025-06-13 | 灌溉水量: 101.31 m³
日期: 2025-06-14 | 灌溉水量: 106.89 m³
日期: 2025-06-16 | 灌溉水量: 100.8 m³
日期: 2025-06-16 | 灌溉水量: 102.12 m³
日期: 2025-06-17 | 灌溉水量: 111.0 m³
日期: 2025-06-17 | 灌溉水量: 102.83 m³
日期: 2025-06-18 | 灌溉水量: 123.58 m³
日期: 2025-06-18 | 灌溉水量: 100.22 m³
日期: 2025-06-20 | 灌溉水量: 101.94 m³
日期: 2025-06-21 | 灌溉水量: 101.03 m³
日期: 2025-06-25 | 灌溉水量: 107.04 m³
日期: 2025-06-25 | 灌溉水量: 100.36 m³
日期: 2025-06-26 | 灌溉水量: 104.03 m³
日期: 2025-06-27 | 灌溉水量: 108.73 m³
日期: 2025-06-28 | 灌溉水量: 114.19 m³
日期: 2025-06-28 | 灌溉水量: 106.45 m³

模拟总灌溉次数: 18
模拟总用水量: 2226.60 m³

```

图 6-1 代码结果图

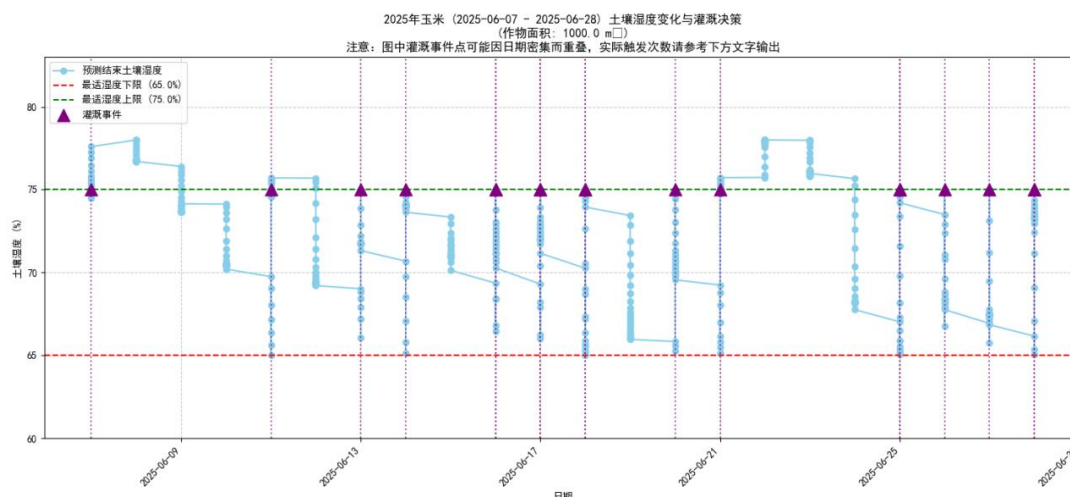


图 6-2 土壤湿度变化和灌溉决策图

该模型在 2025 年 6 月 7 日至 6 月 28 日的玉米抽穗期内，共触发了 18 次灌溉，总用水量为 2226.60 立方米（图 6-1，图 6-2）。

1. 初始阶段高耗水：

模拟从 6 月 7 日开始，由于是模拟灌溉的首日，初始土壤湿度可能较低，导致第一次灌溉量较大(430.84m³)，旨在将土壤湿度迅速提升至适宜范围。

2. 灌溉频率较高：

在模拟的 22 天内(6 月 7 日至 6 月 28 日)，触发了 18 次灌溉，平均不到 1.5 天就灌溉一次。这表明在该模拟条件下(特定气象数据、高 k_c 值、特定土壤参数),水分消耗较快，土壤湿度频繁低于最优下限。

3. 单次灌溉量相对稳定：

除第一次外，后续单次灌溉量大多在 100-120 m³之间，这符合模型将湿度恢复到上限的设计目标，表明每次灌溉都在补充因蒸散发和少量渗漏造成的水分亏缺。

4. 高频率的灌溉表明模型在预测到土壤水分不足时能够及时补充，理论上有助于作物在抽穗期保持健康的生长状态。

七、模型的优缺点

7.1 问题一

优点：

使用指数衰减公式模拟土壤水分变化，合理贴近实际；充分利用历史数据，拟合可靠；可扩展用于不同日期和其他地区预测。

缺点：

未考虑降雨、温度、风速等其他气象变量，可能低估极端天气影响；拟合过程中仅使用线性模型，未探索非线性关系。

7.2 问题二

优点：

1. 多参数耦合，综合考虑了土壤类型、气象条件和作物生长阶段等因素，提高了模型

的准确性和实用性。

2. BAS 算法优化网络初始参数, 克服 BP 神经网络训练时的局部极小问题, 提高训练效率和性能, 采用 BAS-BP 神经网络模型, 能够处理复杂的非线性关系, 对土壤湿度进行较为准确的预测。

3. 误差分析细致, 涵盖晴雨连续段, 便于针对不同气象条件优化模型。

缺点:

1. BAS 算法增加了训练前的计算开销, 其在高维参数空间时优化过程较慢。

2. 模型仍基于人工设定的滞后步长和层数, 缺乏自动特征选择和结构优化。

3. 模型的建立依赖于大量的历史数据, 数据的质量和完整性会影响模型的准确性。

4. 模型假设了一些理想条件, 实际应用中可能存在一定的偏差。

7.3 问题三

优点:

1. 考虑作物阶段性需水: 通过引入作物系数 k_c 来反映玉米不同生长阶段对水分需求的变化, 使得灌溉决策更符合作物实际生理需求。

2. 动态模拟土壤水分平衡: 模型每日更新土壤湿度, 考虑了降雨、蒸散发和灌溉的综合影响, 能够动态反映农田水分状况。

3. 决策阈值明确: 设定了土壤湿度最优上下限, 为灌溉提供了清晰的决策依据, 目标是维持土壤湿度在适宜区间。

4. 可扩展性强: 模型参数(如作物类型、生长阶段、土壤类型、农田面积等)均可调整, 使其适用于不同场景。

缺点:

1. 气象数据依赖性强: 模型的准确性高度依赖于输入气象数据的完整性、准确性和代表性。任何缺失或错误的數據都可能导致预测偏差。

2. 土壤水分运动简化: 模型未深入考虑土壤水分在垂直方向的复杂运动(如毛细管上升、非饱和流等), 仅通过简单的深层渗漏和总水深百分比来模拟, 可能与实际土壤水分分布有所偏差。

3. k_c 值固定: 虽然考虑了作物阶段, 但每个阶段的 k_c 值是固定常数, 未考虑气象条件对的微调影响(如湿度、风速等对蒸腾的影响)。

4. 灌溉策略单一: 目前的灌溉策略是当湿度低于下限时立即补足至上限。实际灌溉可能存在多种策略, 如周期性灌溉、轮灌等, 且可能考虑天气预报、灌溉成本等经济因素。

八、参考文献

- [1] LANE R A, BELL V A, CHAPMAN R M, et al. Evaluating soil moisture simulations from anational-scale gridded hydrological model over Great Britain[J]. Journal of Hydrology: Regional Studies, 2024, 52: 101735.
- [2] LIU Y, CHEN D, MOUATADID S, et al. Development of a daily multilayer cropland soil moisture dataset for China using machine learning and application to cropping patterns[J]. Journal of Hydrometeorology, 2021, 22(2): 445-461.
- [3] LI L, SHANGGUAN W, DENG Y, et al. A causal inference model based on random

- forests to identify the effect of soil moisture on precipitation[J]. Journal of Hydrometeorology, 2020, 21(5): 1115-1131.
- [4] D. Jin and S. Lin (Eds.): Advances in CSIE, Vol. 2, AISC 169, pp. 553–558.
- [5] 孟楚, 李士军, 常晶, 等. 基于 BAS-BP 网络的土壤湿度预测方法研究 [J] . 吉林农业大学学报, 2025, 47 (1) : 179-184.
- [6] Dadari S, Ahmadi S H. Calibration and evaluation of the FAO56-Penman-Monteith, FAO24-radiation, and Priestly-Taylor reference evapotranspiration models using the spatially measured solar radiation across a large arid and semi-arid area in southern Iran[J]. Theoretical & Applied Climatology, 2019: 136(1): 441-455.

附录

问题 1 代码

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from datetime import timedelta
import matplotlib.dates as mdates

import matplotlib.pyplot as plt
from matplotlib import rcParams

rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体，支持中文
rcParams['axes.unicode_minus'] = False # 解决负号 '-' 显示为方块的问题

# ===== 文件路径（请按需修改） =====
ssrd_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\下行辐射强度_ssrd'
soil_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\土壤湿度'

# ===== 初始湿度（%） =====
theta0 = 40

# ===== 读取短波辐射数据（转换为北京时间 UTC+8） =====
def load_hourly_ssrd_data(folder):
    dfs = []
    for file in os.listdir(folder):
        if file.endswith('.xlsx') and 'ssrd' in file and not file.startswith('~$'):
            path = os.path.join(folder, file)
            df = pd.read_excel(path)
            df.columns = ['datetime', 'ssrd']
            df['ssrd'] = df['ssrd'].astype(str).str.replace('W/m²', '').str.strip().astype(float)
            df['datetime'] = pd.to_datetime(df['datetime']) + pd.Timedelta(hours=8) # 转北京时间
            dfs.append(df)
    full_df = pd.concat(dfs)
    full_df.sort_values('datetime', inplace=True)
    return full_df

# ===== 读取土壤湿度数据（转换为北京时间 UTC+8） =====
def load_hourly_soil_data(folder):
    dfs = []
    for file in os.listdir(folder):
```

```

        if (file.endswith('.xlsx') or file.endswith('.csv')) and not file.startswith('~$'):
            path = os.path.join(folder, file)
            df = pd.read_excel(path) if file.endswith('.xlsx') else pd.read_csv(path)
            df.columns = ['datetime', 'soil']
            df['soil'] = df['soil'].astype(str).str.replace('m³/m³',
''.str.strip().astype(float) * 100

            df['datetime'] = pd.to_datetime(df['datetime']) + pd.Timedelta(hours=8) # 转北京时间
            dfs.append(df)

        full_df = pd.concat(dfs)
        full_df.sort_values('datetime', inplace=True)
        return full_df

print("✂ 加载短波辐射数据...")
ssrd_df = load_hourly_ssrd_data(ssrd_folder)
print("✂ 加载土壤湿度数据...")
soil_df = load_hourly_soil_data(soil_folder)

# ===== 拟合每天湿度衰减系数 k 与 白天辐射强度 R (北京时间 6-18 点) =====
results = []
count_days = 0

start_date = soil_df['datetime'].min().floor('D')
end_date = soil_df['datetime'].max().floor('D')

for start_time in pd.date_range(start_date, end_date, freq='24H'):
    end_time = start_time + pd.Timedelta(hours=23)

    soil_sub = soil_df[(soil_df['datetime'] >= start_time) & (soil_df['datetime'] <= end_time)]
    ssrd_sub = ssrd_df[(ssrd_df['datetime'] >= start_time) & (ssrd_df['datetime'] <= end_time)]
    ssrd_daytime = ssrd_sub[(ssrd_sub['datetime'].dt.hour >= 6) & (ssrd_sub['datetime'].dt.hour <=
18)]

    if len(soil_sub) >= 10 and len(ssrd_daytime) >= 5:
        t = np.arange(len(soil_sub))
        theta = soil_sub['soil'].values
        if np.all(theta > 0) and theta[0] > theta[-1]:
            ln_ratio = np.log(theta / theta[0])
            k_fit = -np.polyfit(t, ln_ratio, 1)[0]
            R_avg = ssrd_daytime['ssrd'].mean()
            results.append((R_avg, k_fit))
            count_days += 1

print(f"\n✂ 共处理 {count_days} 天数据。")

```

```

results = np.array(results)
R_all = results[:, 0].reshape(-1, 1)
k_all = results[:, 1]

reg = LinearRegression().fit(R_all, k_all)
a, b = reg.coef_[0], reg.intercept_

print(f"\n✓ 线性拟合完成: k = {a:.8f} * R + {b:.8f}")

# ===== 预测未来 48 小时湿度变化 (2025-07-12 和 07-13, 北京时间) =====
def predict_R_for_date(target_date, ssrd_df):
    R_list = []
    for year in range(2021, 2025):
        hist_date = pd.to_datetime(f"{year}-{target_date.month:02}-{target_date.day:02}")
        ssrd_hist = ssrd_df[(ssrd_df['datetime'].dt.date == hist_date.date())]
        ssrd_daytime = ssrd_hist[(ssrd_hist['datetime'].dt.hour >= 6) &
(ssrd_hist['datetime'].dt.hour <= 18)]
        if not ssrd_daytime.empty:
            R_list.append(ssrd_daytime['ssrd'].mean())
    if len(R_list) == 0:
        return None
    return np.mean(R_list)

def get_avg_soil_curve(month, day, soil_df):
    """ 获取指定月日 (7 月 12 日或 7 月 13 日) 2021-2024 年同期小时平均土壤湿度曲线 """
    curves = []
    for year in range(2021, 2025):
        day_start = pd.to_datetime(f"{year}-{month:02}-{day:02}")
        day_end = day_start + timedelta(hours=23)
        soil_sub = soil_df[(soil_df['datetime'] >= day_start) & (soil_df['datetime'] <= day_end)]
        if len(soil_sub) == 24:
            curves.append(soil_sub['soil'].values)
    if curves:
        avg_curve = np.mean(curves, axis=0) # 24 小时平均
        return avg_curve
    else:
        return None

theta_pred_all = []
time_index = []

for day_offset in [0, 1]: # 7 月 12、13 日
    date_target = pd.to_datetime('2025-07-12') + timedelta(days=day_offset)

```

```

R_est = predict_R_for_date(date_target, ssrd_df)

if R_est is None:
    print(f"✗ 无历史数据，无法预测 {date_target.date()} 的辐射强度")
    continue

k_est = a * R_est + b
print(f"  预测 {date_target.date()} (北京时间): R = {R_est:.2f} W/m², k = {k_est:.5f} /h")

for hour in range(24):
    t = hour
    theta = theta0 * np.exp(-k_est * t)
    theta_pred_all.append(theta)
    time_index.append(date_target + timedelta(hours=hour)) # 北京时间直接使用

# ===== 可视化结果（双子图）=====
import matplotlib.gridspec as gridspec

plt.figure(figsize=(10, 8))
gs = gridspec.GridSpec(2, 1, height_ratios=[2, 1])

# 子图 1: k vs R 拟合
ax0 = plt.subplot(gs[0])
ax0.scatter(R_all, k_all, s=25, alpha=0.7, label='样本点')
ax0.plot(R_all, reg.predict(R_all), color='red', label=f'拟合线: k = {a:.8f}R + {b:.8f}')
ax0.set_xlabel('白天平均短波辐射强度 R (W/m²)', fontsize=12)
ax0.set_ylabel('湿度衰减系数 k (1/h)', fontsize=12)
ax0.set_title('k = f(R) 拟合结果', fontsize=13)
ax0.legend()
ax0.grid(True)

# 子图 2: 模拟不同 k 的湿度变化  $\theta(t)$ 
ax1 = plt.subplot(gs[1])
t_sim = np.linspace(0, 24, 100)
theta0_sim = theta0
k_vals = np.percentile(k_all, [20, 50, 80]) # 使用三种典型 k 值（慢速/中速/快速衰减）

colors = ['blue', 'green', 'orange']
for i, k_sim in enumerate(k_vals):
    theta_sim = theta0_sim * np.exp(-k_sim * t_sim)
    ax1.plot(t_sim, theta_sim, color=colors[i], label=f'k = {k_sim:.4f}')

ax1.set_xlabel('时间 t (小时)', fontsize=12)
ax1.set_ylabel('湿度  $\theta(t)$  (%)', fontsize=12)

```

```

ax1.set_title(r'湿度衰减曲线  $\theta(t) = \theta_0 e^{-kt}$  (典型  $k$  值)', fontsize=13)

ax1.legend()
ax1.grid(True)

plt.tight_layout()
plt.show()

# 绘制预测曲线 + 历史同期平均曲线
plt.figure(figsize=(12, 5))

# 预测曲线
plt.plot(time_index, theta_pred_all, marker='o', color='purple', label='2025 年预测湿度')

# 历史同期曲线 (2021-2024 年 7 月 12 日和 7 月 13 日)
for day_offset in [0, 1]:
    month, day = 7, 12 + day_offset
    avg_curve = get_avg_soil_curve(month, day, soil_df)
    if avg_curve is not None:
        # 生成时间索引 (北京时间)
        base_date = pd.to_datetime(f"2025-{month:02}-{day:02}")
        time_idx_hist = [base_date + timedelta(hours=h) for h in range(24)]
        plt.plot(time_idx_hist, avg_curve, linestyle='--', label=f'2021-2024 年平均 {month}-{day}')

plt.xlabel('北京时间', fontsize=12)
plt.ylabel('湿度  $\theta(t)$  (%)', fontsize=12)
plt.title('土壤湿度预测与历史同期平均对比 (2025 年 7 月 12-13 日)', fontsize=14)
plt.legend()
plt.grid(True)

plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%m-%d %H:%M'))
plt.gcf().autofmt_xdate()
plt.tight_layout()
plt.show()

```

问题 2 代码

```

import os
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, mean_squared_error
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import random # 用于 BAS 算法的随机性

```

```

# --- 在这里添加字体设置 ---
import matplotlib.font_manager as fm
plt.rcParams['font.sans-serif'] = ['SimHei'] # 尝试使用 SimHei 字体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
# --- 字体设置结束 ---

# ... 后续的数据加载、处理、模型定义和训练代码 ...

# ===== 数据路径（保持不变） =====
wind_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\风速'
rain_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\降水量'
temp_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\温度数据'
soil_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\土壤湿度'
ssrd_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\下行辐射强度_ssrd'

# ===== 通用数据读取函数（保持不变） =====
def load_data(folder, col_name, unit, scale=1.0):
    """
    读取指定文件夹内所有 xlsx 文件的数据，提取两列：
    时间、对应气象变量列，清洗单位，转换为 float，统一放入一个 DataFrame。
    """
    dfs = []
    for file in os.listdir(folder):
        if file.endswith('.xlsx') and not file.startswith('~$'):
            path = os.path.join(folder, file)
            df = pd.read_excel(path)
            df.columns = ['datetime', col_name]
            df[col_name] = df[col_name].astype(str).str.replace(unit,
''.strip()).astype(float) * scale
            df['datetime'] = pd.to_datetime(df['datetime'])
            dfs.append(df)
    return pd.concat(dfs).sort_values('datetime').reset_index(drop=True)

# ===== 加载所有数据（保持不变） =====
wind_df = load_data(wind_folder, 'wind', 'm/s')
rain_df = load_data(rain_folder, 'rain', 'mm')
temp_df = load_data(temp_folder, 'temp', '°C')
soil_df = load_data(soil_folder, 'soil', 'm³/m³', scale=100) # 土壤湿度单位转为百分比
ssrd_df = load_data(ssrd_folder, 'ssrd', 'W/m²')

# ===== 合并所有数据（保持不变） =====
df_all = wind_df.merge(rain_df, on='datetime', how='inner') \
    .merge(temp_df, on='datetime', how='inner') \

```

```

        .merge(ssrd_df, on='datetime', how='inner') \
        .merge(soil_df, on='datetime', how='left')

# ===== 只保留 6-7 月数据（保持不变） =====
df_all = df_all[df_all['datetime'].dt.month.isin([6, 7])].reset_index(drop=True)

# ===== 引入时序特征（新增部分） =====
# 提取年份，方便后续分组分析（保持不变）
df_all['year'] = df_all['datetime'].dt.year

# 定义要进行滞后操作的特征和滞后步长
features_to_lag = ['wind', 'rain', 'temp', 'ssrd', 'soil']
lags = [1, 3, 6] # 滞后 1 小时、3 小时、6 小时

for feature in features_to_lag:
    for lag in lags:
        df_all[f'{feature}_t-{lag}'] = df_all[feature].shift(lag)

# 移除由于滞后操作而产生的 NaN 行
df_all.dropna(inplace=True)
df_all.reset_index(drop=True, inplace=True)

# 重新构造训练特征与标签
# 原始特征
current_features = ['wind', 'rain', 'temp', 'ssrd']
# 滞后特征
lagged_feature_names = [f'{f}_t-{l}' for f in features_to_lag for l in lags]

X_raw = df_all[current_features + lagged_feature_names] # 输入特征：当前和滞后气象变量、滞后土壤湿度
y_raw = df_all['soil'] # 标签：当前土壤湿度

print(f"原始特征数量：{len(current_features)}")
print(f"滞后特征数量：{len(lagged_feature_names)}")
print(f"总输入特征数量：{X_raw.shape[1]}")
print(f"经过滞后处理和 NaN 移除后，数据样本数量：{df_all.shape[0]}")

# ===== 标准化（y_scaled 保持为 2D 数组） =====
scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_scaled = scaler_X.fit_transform(X_raw.values) # 将 X_raw 转换为 NumPy 数组再拟合
y_scaled = scaler_y.fit_transform(y_raw.values.reshape(-1, 1))

# ===== 自定义 BP 神经网络实现（引入 Adam 优化器，无变化） =====

```

```

class CustomBPNeuralNetwork:
    def __init__(self, input_size, hidden_sizes, output_size, activation='relu',
                  learning_rate=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8):
        self.input_size = input_size
        self.hidden_sizes = hidden_sizes
        self.output_size = output_size
        self.activation = activation
        self.learning_rate = learning_rate
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.weights = []
        self.biases = []
        self.loss_curve = [] # 记录训练损失
        self.m_w, self.v_w = [], []
        self.m_b, self.v_b = [], []
        self.t = 0 # time step for Adam

    def _initialize_weights_biases(self, initial_params=None):
        layer_sizes = [self.input_size] + list(self.hidden_sizes) + [self.output_size]

        if initial_params is not None:
            param_idx = 0
            for i in range(len(layer_sizes) - 1):
                w_shape = (layer_sizes[i], layer_sizes[i+1])
                b_shape = (layer_sizes[i+1],)

                num_weights = w_shape[0] * w_shape[1]
                num_biases = b_shape[0]

                self.weights.append(initial_params[param_idx : param_idx +
num_weights].reshape(w_shape))
                param_idx += num_weights

                self.biases.append(initial_params[param_idx : param_idx +
num_biases].reshape(b_shape))
                param_idx += num_biases
            else:
                for i in range(len(layer_sizes) - 1):
                    w = np.random.randn(layer_sizes[i], layer_sizes[i+1]) * np.sqrt(2.0 /
layer_sizes[i])
                    self.weights.append(w)
                    b = np.zeros((layer_sizes[i+1],))
                    self.biases.append(b)

```



```

        self.m_w = [np.zeros_like(w) for w in self.weights]
        self.v_w = [np.zeros_like(w) for w in self.weights]
        self.m_b = [np.zeros_like(b) for b in self.biases]
        self.v_b = [np.zeros_like(b) for b in self.biases]
        self.t = 0

    def _relu(self, x):
        return np.maximum(0, x)

    def _relu_derivative(self, x):
        return (x > 0).astype(float)

    def _forward(self, X):
        self.activations = [X]
        self.z_values = []

        a = X
        for i in range(len(self.weights)):
            z = np.dot(a, self.weights[i]) + self.biases[i]
            self.z_values.append(z)
            if i == len(self.weights) - 1:
                a = z
            else:
                a = self._relu(z)
            self.activations.append(a)
        return a

    def _backward(self, X, y_true, y_pred):
        grads_w = [np.zeros_like(w) for w in self.weights]
        grads_b = [np.zeros_like(b) for b in self.biases]

        delta = y_pred - y_true

        for i in reversed(range(len(self.weights))):
            grad_w = np.dot(self.activations[i].T, delta)
            grad_b = np.sum(delta, axis=0)

            grads_w[i] = grad_w
            grads_b[i] = grad_b

            if i != 0:
                delta = np.dot(delta, self.weights[i].T) *
self._relu_derivative(self.z_values[i-1])

```

```

        return grads_w, grads_b

def train(self, X, y, epochs=100, batch_size=32, validation_split=0.1, verbose=False):
    num_samples = X.shape[0]
    val_samples = int(num_samples * validation_split)
    train_X, val_X = X[val_samples:], X[:val_samples]
    train_y, val_y = y[val_samples:], y[:val_samples]

    n_batches_train = len(train_X) // batch_size

    best_val_loss = float('inf')
    epochs_no_improve = 0
    patience = 3000 # 早停机制的耐心值, 连续多少个 epoch 验证损失没有改善就停止,
                    # 设置为 3000 相当于没有早停

    for epoch in range(epochs):
        self.t += 1

        permutation = np.random.permutation(len(train_X))
        shuffled_X = train_X[permutation]
        shuffled_y = train_y[permutation]

        epoch_train_loss = 0
        for i in range(n_batches_train):
            start = i * batch_size
            end = start + batch_size
            X_batch = shuffled_X[start:end]
            y_batch = shuffled_y[start:end]

            y_pred = self._forward(X_batch)

            loss = np.mean((y_pred - y_batch)**2)
            epoch_train_loss += loss

        grads_w, grads_b = self._backward(X_batch, y_batch, y_pred)

        for j in range(len(self.weights)):
            self.m_w[j] = self.beta1 * self.m_w[j] + (1 - self.beta1) * grads_w[j]
            self.v_w[j] = self.beta2 * self.v_w[j] + (1 - self.beta2) * (grads_w[j]**2)
            m_w_hat = self.m_w[j] / (1 - self.beta1**self.t)
            v_w_hat = self.v_w[j] / (1 - self.beta2**self.t)
            self.weights[j] -= self.learning_rate * m_w_hat / (np.sqrt(v_w_hat) +
self.epsilon)

```

```

        self.m_b[j] = self.beta1 * self.m_b[j] + (1 - self.beta1) * grads_b[j]
        self.v_b[j] = self.beta2 * self.v_b[j] + (1 - self.beta2) * (grads_b[j]**2)
        m_b_hat = self.m_b[j] / (1 - self.beta1**self.t)
        v_b_hat = self.v_b[j] / (1 - self.beta2**self.t)
        self.biases[j] -= self.learning_rate * m_b_hat / (np.sqrt(v_b_hat) + self.epsilon)

    avg_train_loss = epoch_train_loss / n_batches_train
    self.loss_curve.append(avg_train_loss)

    if len(val_X) > 0:
        val_pred = self._forward(val_X)
        val_loss = np.mean((val_pred - val_y)**2)
        if verbose:
            print(f"Epoch {epoch+1}/{epochs}, Train Loss: {avg_train_loss:.4f}, Val Loss: {val_loss:.4f}")

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            epochs_no_improve = 0
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:
                if verbose:
                    print(f"Early stopping at epoch {epoch+1}")
                break
    else:
        if verbose:
            print(f"Epoch {epoch+1}/{epochs}, Train Loss: {avg_train_loss:.4f}")

def predict(self, X):
    return self._forward(X)

def get_flattened_params(self):
    params = []
    for w in self.weights:
        params.extend(w.flatten())
    for b in self.biases:
        params.extend(b.flatten())
    return np.array(params)

def set_flattened_params(self, new_params):
    param_idx = 0
    for i in range(len(self.weights)):

```

```

        w_shape = self.weights[i].shape
        num_weights = w_shape[0] * w_shape[1]
        self.weights[i] = new_params[param_idx : param_idx + num_weights].reshape(w_shape)
        param_idx += num_weights

        b_shape = self.biases[i].shape
        num_biases = b_shape[0]
        self.biases[i] = new_params[param_idx : param_idx + num_biases].reshape(b_shape)
        param_idx += num_biases

# ===== BAS 算法（无变化） =====
def bas_optimize_initial_params(X_train, y_train, input_size, hidden_sizes, output_size,
                                max_bas_iterations=100, initial_delta=10, eta=0.95):

    temp_bp_net = CustomBPNeuralNetwork(input_size, hidden_sizes, output_size)
    temp_bp_net._initialize_weights_biases()
    k = len(temp_bp_net.get_flattened_params())

    delta = initial_delta
    best_fitness = float('inf')
    best_params = np.random.uniform(-0.5, 0.5, k)

    for iter_bas in range(max_bas_iterations):
        b_vec = np.random.randn(k)
        b_vec = b_vec / np.linalg.norm(b_vec)

        x_left = best_params - 0.5 * delta * b_vec
        x_right = best_params + 0.5 * delta * b_vec

        temp_bp_net.set_flattened_params(x_right)
        y_pred_right = temp_bp_net.predict(X_train)
        fitness_right = np.mean((y_pred_right - y_train)**2)

        temp_bp_net.set_flattened_params(x_left)
        y_pred_left = temp_bp_net.predict(X_train)
        fitness_left = np.mean((y_pred_left - y_train)**2)

        if fitness_right < fitness_left:
            best_params = best_params - delta * b_vec
        else:
            best_params = best_params + delta * b_vec

    delta = delta * eta

```

```

        current_fitness = min(fitness_left, fitness_right)
        if current_fitness < best_fitness:
            best_fitness = current_fitness

        if iter_bas % 10 == 0 or iter_bas == max_bas_iterations - 1:
            print(f"BAS Iter {iter_bas+1}/{max_bas_iterations}, Current Best Fitness:
{best_fitness:.6f}, Delta: {delta:.6f}")
            if best_fitness < 0.0001:
                print(f"BAS Optimization converged at iteration {iter_bas+1}.")
                break

        print(f"BAS Optimization Finished. Final Best Initial MSE: {best_fitness:.4f}")
        return best_params

# ===== BAS-BP 神经网络训练（优化：尝试更大的网络结构和调整超参数） =====
# 定义网络结构
input_neurons = X_scaled.shape[1]
# 1. 网络结构改为 (128, 64, 32)
hidden_neurons = (128, 64, 32)
output_neurons = 1

print("\n==== 开始 BAS-BP 神经网络优化训练 ==== \n")
print("开始 BAS 算法优化 BP 神经网络初始参数...")
initial_weights_biases = bas_optimize_initial_params(X_scaled, y_scaled,
                                                    input_size=input_neurons,
                                                    hidden_sizes=hidden_neurons,
                                                    output_size=output_neurons,
                                                    max_bas_iterations=500, # 可以根据需要调整
                                                    initial_delta=0.5,
                                                    eta=0.98)

# 使用 BAS 优化后的初始参数构建并训练 BP 神经网络
print("\n开始训练 BAS-BP 神经网络（使用 Adam 优化器和调整后的网络结构）...")
bas_bp_model = CustomBPNeuralNetwork(input_neurons, hidden_neurons, output_neurons,
learning_rate=0.001)
bas_bp_model._initialize_weights_biases(initial_params=initial_weights_biases)
# 2. 训练 epochs 提高至 3000, 添加 early stopping（已在 CustomBPNeuralNetwork 中实现）
bas_bp_model.train(X_scaled, y_scaled, epochs=10000, batch_size=64, validation_split=0.1,
verbose=True)

# ===== 预测结果还原（无变化） =====
y_pred_scaled = bas_bp_model.predict(X_scaled)
y_pred = scaler_y.inverse_transform(y_pred_scaled).ravel()

```

```

df_all['soil_pred'] = y_pred

# ===== 训练损失曲线（无变化） =====
plt.figure(figsize=(8,4))
plt.plot(bas_bp_model.loss_curve)
plt.xlabel('迭代次数')
plt.ylabel('损失值 (MSE)')
plt.title('BAS-BP 训练损失曲线 (Adam 优化器)')
plt.grid(True)
plt.show()

# ===== 误差评估（无变化） =====
print(f'\n 整体 RMSE: {root_mean_squared_error(y_raw, y_pred):.4f}')
print(f'整体 MAE: {mean_absolute_error(y_raw, y_pred):.4f}')

# 3. 画出预测误差的分布直方图
errors = y_raw - y_pred
plt.figure(figsize=(10, 6))
plt.hist(errors, bins=50, edgecolor='black')
plt.title('预测误差分布直方图')
plt.xlabel('预测误差 (真实值 - 预测值)')
plt.ylabel('频数')
plt.grid(True)
plt.show()

# ===== 每年输出对比图（无变化） =====
for year in sorted(df_all['year'].unique()):
    df_year = df_all[df_all['year'] == year].reset_index(drop=True)
    plt.figure(figsize=(14, 5))
    plt.plot(df_year['datetime'], df_year['soil'], label='真实土壤湿度', color='blue')
    plt.plot(df_year['datetime'], df_year['soil_pred'], label='预测土壤湿度', color='red')
    plt.title(f'{year}年 6-7 月土壤湿度预测对比 (BAS-BP)')
    plt.xlabel('时间')
    plt.ylabel('土壤湿度 (%)')
    plt.legend()
    plt.grid(True)
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%m-%d'))
    plt.gca().xaxis.set_major_locator(mdates.DayLocator(interval=5))
    plt.gcf().autofmt_xdate()
    plt.tight_layout()
    plt.show()

# ===== 连续晴天/雨天误差计算函数（保持不变） =====

```

```

def calc_weather_period_errors(df, rain_col='rain', true_col='soil', pred_col='soil_pred',
threshold_hours=12):
    import numpy as np
    from sklearn.metrics import mean_squared_error, mean_absolute_error

    df = df.copy()
    df = df.sort_values('datetime').reset_index(drop=True)

    df['is_rain'] = df[rain_col] > 0

    df['rain_shift'] = df['is_rain'].shift(1)
    df['period_change'] = df['is_rain'] != df['rain_shift']
    df['period_id'] = df['period_change'].cumsum()

    results = {
        '晴天段': [],
        '雨天段': []
    }

    for period_id, group in df.groupby('period_id'):
        length = len(group)
        is_rain = group['is_rain'].iloc[0]
        if length >= threshold_hours:
            true_vals = group[true_col].values
            pred_vals = group[pred_col].values
            rmse = np.sqrt(mean_squared_error(true_vals, pred_vals))
            mae = mean_absolute_error(true_vals, pred_vals)
            period_info = {
                'start_time': group['datetime'].iloc[0],
                'end_time': group['datetime'].iloc[-1],
                'hours': length,
                'rmse': rmse,
                'mae': mae
            }
            if is_rain:
                results['雨天段'].append(period_info)
            else:
                results['晴天段'].append(period_info)

    def summarize(period_list):
        if len(period_list) == 0:
            return {'rmse_mean': None, 'mae_mean': None, 'count': 0}
        rmse_mean = np.mean([p['rmse'] for p in period_list])
        mae_mean = np.mean([p['mae'] for p in period_list])

```

```

        return {'rmse_mean': rmse_mean, 'mae_mean': mae_mean, 'count': len(period_list)}

    sunny_summary = summarize(results['晴天段'])
    rainy_summary = summarize(results['雨天段'])

    print("\n 连续晴天段数:", sunny_summary['count'])
    print(f"晴天平均 RMSE: {sunny_summary['rmse_mean']:.4f}, 平均 MAE: {sunny_summary['mae_mean']:.4f}")
    print("连续雨天段数:", rainy_summary['count'])
    print(f"雨天平均 RMSE: {rainy_summary['rmse_mean']:.4f}, 平均 MAE: {rainy_summary['mae_mean']:.4f}")

    return results

# ===== 调用连续晴雨天误差计算（无变化） =====
weather_period_errors = calc_weather_period_errors(df_all, threshold_hours=12)

```

问题 3 代码

```

import pandas as pd
import numpy as np
import os
import re
import matplotlib.pyplot as plt
import matplotlib as mpl # 导入 matplotlib 模块本身
import matplotlib.pyplot as plt
import matplotlib as mpl

mpl.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial Unicode MS', 'DejaVu Sans']
mpl.rcParams['axes.unicode_minus'] = False # 解决负号 '-' 显示为方块的问题

# ===== 通用数据读取函数（已优化，更健壮地处理数据清洗） =====
def load_data(folder, col_name, unit, scale=1.0):
    """
    读取指定文件夹内所有 xlsx 文件的数据，提取两列：
    时间、对应气象变量列，清洗单位，转换为 float，统一放入一个 DataFrame。
    """
    dfs = []
    for file in os.listdir(folder):
        if file.endswith('.xlsx') and not file.startswith('~$'):
            path = os.path.join(folder, file)
            df = pd.read_excel(path)

```



```

# 确保 DataFrame 有足够的列，避免索引错误
if df.shape[1] < 2:
    print(f"警告：文件 {file} 列数不足，可能为空或格式错误。跳过。")
    continue

# 假设第一列是时间，第二列是数据
# 尝试查找包含“时间”或“日期”的列作为时间列
time_col_candidates = [c for c in df.columns if '时间' in str(c) or '日期' in str(c)]
# 尝试查找包含变量名或数值的列作为数据列
data_col_candidates = [c for c in df.columns if col_name in str(c) or df[c].apply(lambda
x: isinstance(x, (int, float)) or (isinstance(x, str) and re.match(r'([-+]?\d*\.\d+)', x))).any()]

if time_col_candidates and data_col_candidates:
    # 优先使用检测到的列，否则回退到默认的[0]和[1]
    time_col = time_col_candidates[0]
    data_col = data_col_candidates[0]
else:
    # 如果自动检测失败，回退到默认的第一列和第二列
    print(f"警告：文件 {file} 未能自动识别时间或数据列，尝试使用默认前两列。")
    time_col = df.columns[0]
    data_col = df.columns[1]

# 重名列以方便后续处理
df.columns = ['datetime_raw' if c == time_col else 'data_raw' if c == data_col else c
for c in df.columns]

# 尝试将时间列转换为 datetime，如果失败则该行转换为 NaT，后续会丢弃
df['datetime'] = pd.to_datetime(df['datetime_raw'], errors='coerce')

# 健壮地清洗数据列：
# 1. 确保是字符串类型
# 2. 使用正则表达式提取所有数字、小数点和正负号。这将忽略任何单位符号。
df['clean_data'] = df['data_raw'].astype(str).str.extract(r'([-+]?\d*\.\d+)',
expand=False)

# 将提取出的字符串转换为数值类型，无法转换的变为 NaN
df[col_name] = pd.to_numeric(df['clean_data'], errors='coerce') * scale

# 筛选掉时间或数据转换失败的行
df = df.dropna(subset=['datetime', col_name])

if not df.empty: # 只有当 DataFrame 非空时才添加
    dfs.append(df[['datetime', col_name]])
else:

```

```

        print(f"警告：文件 {file} 在清洗后没有有效数据。")

    if not dfs:
        print(f"错误：未能从文件夹 {folder} 加载任何有效数据。请检查文件和路径。")
        return pd.DataFrame() # 返回空 DataFrame
    return pd.concat(dfs).sort_values('datetime').reset_index(drop=True)

# ===== 文件夹路径定义 =====
wind_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\风速'
rain_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\降水量'
temp_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\温度数据'
soil_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\土壤湿度'
ssrd_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\下行辐射强度_ssrd'
dew_point_folder = r'C:\Users\32610\Desktop\shuxuejianmo\数据\露点温度'

# ===== 加载所有数据 =====
print("正在加载风速数据...")
wind_df = load_data(wind_folder, 'wind', 'm/s')
print("正在加载降水量数据...")
rain_df = load_data(rain_folder, 'rain', 'mm')
print("正在加载温度数据...")
temp_df = load_data(temp_folder, 'temp', '°C')
print("正在加载土壤湿度数据...")
soil_df = load_data(soil_folder, 'soil', 'm³/m³', scale=100) # 土壤湿度单位转为百分比
print("正在加载下行辐射强度数据...")
ssrd_df = load_data(ssrd_folder, 'ssrd', 'W/m²')
print("正在加载露点温度数据...")
dew_point_df = load_data(dew_point_folder, 'dew_point', '°C') # 加载露点温度数据

# 检查关键数据是否成功加载
if wind_df.empty or rain_df.empty or temp_df.empty or ssrd_df.empty or dew_point_df.empty:
    print("错误：某些关键气象数据加载失败或为空，无法进行后续计算。请检查数据文件和路径。")
    exit() # 退出程序

# ===== 合并所有数据 =====
print("正在合并所有数据...")
df_all = wind_df.merge(rain_df, on='datetime', how='inner') \
    .merge(temp_df, on='datetime', how='inner') \
    .merge(ssrd_df, on='datetime', how='inner') \
    .merge(dew_point_df, on='datetime', how='inner') \
    .merge(soil_df, on='datetime', how='left') # 土壤湿度可能不是每天都有

# 去除重复的 datetime，保留第一个，并按 datetime 排序

```

```

df_all =
df_all.drop_duplicates(subset=['datetime']).sort_values('datetime').reset_index(drop=True)

# ===== IMPORTANT: Filter for 2025 data only =====
print("正在筛选 2025 年的数据...")
df_all_2025 = df_all[df_all['datetime'].dt.year == 2025].reset_index(drop=True)

if df_all_2025.empty:
    print("\n 警告：在您提供的原始数据文件中，未找到任何 2025 年的气象数据。")
    print("本模拟将**无法**针对 2025 年进行计算。请确保您的 Excel 文件中包含 2025 年的数据（例如，数据时间戳为 2025 年）。")
    exit() # 如果没有 2025 年数据，则退出

df_all = df_all_2025 # 使用 2025 年过滤后的数据进行后续操作

# 确保 'datetime' 列是 datetime 格式
df_all['datetime'] = pd.to_datetime(df_all['datetime'])

# ===== 设定模拟日期范围（与灌溉日期范围一致） =====
data_year_sim = 2025 # 显式设定模拟年份为 2025
simulation_start = pd.Timestamp(f'{data_year_sim}-06-07') # 从 6 月 7 日开始模拟
simulation_end = pd.Timestamp(f'{data_year_sim}-06-28') # 模拟到 6 月 28 日

# 筛选出模拟期的数据
df_simulation_period = df_all[(df_all['datetime'].dt.date >= simulation_start.date()) & \
                               (df_all['datetime'].dt.date <= simulation_end.date())].copy()

if df_simulation_period.empty:
    print(f"错误：在 2025 年的数据中，未找到 {simulation_start.date()} 至 {simulation_end.date()} 期间\n的数据。无法进行模拟。")
    exit()

# --- 灌溉决策参数 ---
OPTIMAL_MOISTURE_LOWER_BOUND = 65.0 # %
OPTIMAL_MOISTURE_UPPER_BOUND = 75.0 # %
FIELD_AREA_SQ_METERS = 1000.0 # 修改为 1000 平方米
CORN_ROOT_DEPTH_METERS = 1.0 # 示例值：1 米，请根据农学数据或具体玉米品种调整

# --- 物理模型参数 ---
FIELD_CAPACITY = 78.0 # 土壤田间持水量（%），请根据您的土壤类型调整

# --- 作物阶段及对应 Kc 值 ---
# 根据您的要求，抽穗期为 2025 年 6 月 7 日至 6 月 28 日
CORN_GROWTH_STAGES = [

```

```

    {'stage': '初始阶段', 'start': '2025-05-15', 'end': '2025-06-06', 'Kc': 0.3}, # 调整结束日期
    {'stage': '中期阶段（抽穗期）', 'start': '2025-06-07', 'end': '2025-06-28', 'Kc': 1.20}, # 用户
要求：抽穗期
    {'stage': '晚期阶段', 'start': '2025-06-29', 'end': '2025-08-15', 'Kc': 0.6}, # 调整起始日期
]

# 将日期字符串转换为 Timestamp 对象以便比较
for stage in CORN_GROWTH_STAGES:
    stage['start'] = pd.Timestamp(stage['start'])
    stage['end'] = pd.Timestamp(stage['end'])

# --- Penman-Monteith 所需的站点及常数（遵循 FAO-56 方法） ---
ELEVATION_M = 5.0 # 气象站海拔（米）

# 纬度（度）。FAO-56 Penman-Monteith 需要，用于计算理论晴空辐射和日照时数。
# 这里使用无锡值，北纬 31 度。
# 请根据您的实际农田位置调整此值。
LATITUDE_DEG = 31.0 # 北纬 31 度

# 常数
GSC = 0.082 # 太阳常数 (MJ m-2 min-1)
SIGMA = 4.903e-9 # 斯特芬-玻尔兹曼常数 (MJ K-4 m-2 day-1)
LAMBDA_WATER = 2.45 # 水的汽化潜热 (MJ kg-1)

# --- 辅助函数：计算饱和水汽压 (kPa) ---
def es_calc(T_celsius):
    return 0.6108 * np.exp(17.27 * T_celsius / (T_celsius + 237.3))

# --- 辅助函数：计算平均饱和水汽压 (kPa) ---
def mean_es(T_celsius):
    return es_calc(T_celsius)

# --- 辅助函数：计算理论晴空辐射 (Rso, MJ m-2 day-1) ---
def calculate_rso(day_of_year, latitude_deg, elevation_m):
    lat_rad = np.deg2rad(latitude_deg)
    delta_solar = 0.409 * np.sin(2 * np.pi * day_of_year / 365 - 1.39)
    omega_s = np.arccos(-np.tan(lat_rad) * np.tan(delta_solar))
    Ra = (24 * 60 / np.pi) * GSC * ( (omega_s * np.sin(lat_rad) * np.sin(delta_solar)) + \
                                     (np.cos(lat_rad) * np.cos(delta_solar) * np.sin(omega_s)) )
    Rso = (0.75 + 2e-5 * elevation_m) * Ra
    return Rso

# --- 获取初始土壤湿度作为预测起点 ---
# 在模拟开始日期的土壤湿度数据，如果没有，则使用默认值 70%
initial_soil_data = df_simulation_period[df_simulation_period['datetime'].dt.date ==

```

```

simulation_start.date())['soil']
INITIAL_SOIL_MOISTURE = initial_soil_data.iloc[0] if not initial_soil_data.empty and not
pd.isna(initial_soil_data.iloc[0]) else 70.0

# --- 预测模拟 ---
predicted_soil_moisture = INITIAL_SOIL_MOISTURE
simulated_decisions = []

print(f"\n 模拟开始时的初始土壤湿度（预测起点）: {INITIAL_SOIL_MOISTURE:.2f}%")

# 定义灌溉限定日期范围（现在与模拟范围一致）
IRRIGATION_START_DATE = simulation_start.date()
IRRIGATION_END_DATE = simulation_end.date()
print(f"灌溉决策将只在 {IRRIGATION_START_DATE} 至 {IRRIGATION_END_DATE} 期间进行判断和触发。")

# 遍历模拟期数据，进行每日预测和决策
for index, row in df_simulation_period.iterrows():
    current_date = row['datetime']

    # 确定当前日期的作物系数 Kc
    KC_CORN_CURRENT_DAY = None
    current_stage_name = '未知阶段'
    for stage_info in CORN_GROWTH_STAGES:
        # 使用 .date() 确保只比较日期部分
        if stage_info['start'].date() <= current_date.date() <= stage_info['end'].date():
            KC_CORN_CURRENT_DAY = stage_info['Kc']
            current_stage_name = stage_info['stage']
            break

    if KC_CORN_CURRENT_DAY is None:
        # 即使跳过预测，也应该更新 simulated_decisions 列表以保持日期完整性
        simulated_decisions.append({
            '日期': current_date.date(),
            '作物阶段': '未知阶段', # 保持列名一致
            '当前 Kc 值': np.nan,
            '当日气温 (°C)': round(row['temp'], 2),
            '当日露点温度 (°C)': round(row['dew_point'], 2) if pd.notna(row['dew_point']) else
np.nan,
            '当日风速 (m/s)': round(row['wind'], 2) if pd.notna(row['wind']) else np.nan,
            '当日降雨量 (mm)': round(row['rain'], 2) if pd.notna(row['rain']) else np.nan,
            '当日辐射 (W/m²)': round(row['ssrd'], 2) if pd.notna(row['ssrd']) else np.nan,
            '预测起始湿度 (%)': predicted_soil_moisture, # 使用前一天的结束湿度作为起始点
            '预测蒸散发 (mm)': np.nan,

```

```

        '预测结束湿度 (%)': predicted_soil_moisture, # 未预测, 湿度保持不变
        '行动': '未定义作物阶段, 无法预测',
        '建议灌溉水量 (m³)': 0
    })
    continue

# 获取当天气象数据
temp = row['temp'] # 平均气温 °C
wind = row['wind'] # 风速 m/s
rain = row['rain'] # 降雨量 mm
ssrd = row['ssrd'] # 下行短波辐射强度 W/m²
dew_point = row['dew_point'] # 露点温度 °C

# 检查气象数据完整性
if pd.isna(temp) or pd.isna(wind) or pd.isna(rain) or pd.isna(ssrd) or pd.isna(dew_point):
    simulated_decisions.append({
        '日期': current_date.date(),
        '作物阶段': current_stage_name, # 保持列名一致
        '当前 Kc 值': round(KC_CORN_CURRENT_DAY, 2),
        '当日气温 (°C)': round(temp, 2) if pd.notna(temp) else np.nan,
        '当日露点温度 (°C)': round(dew_point, 2) if pd.notna(dew_point) else np.nan,
        '当日风速 (m/s)': round(wind, 2) if pd.notna(wind) else np.nan,
        '当日降雨量 (mm)': round(rain, 2) if pd.notna(rain) else np.nan,
        '当日辐射 (W/m²)': round(ssrd, 2) if pd.notna(ssrd) else np.nan,
        '预测起始湿度 (%)': predicted_soil_moisture, # 使用前一天的结束湿度作为起始点
        '预测蒸散发 (mm)': np.nan,
        '预测结束湿度 (%)': predicted_soil_moisture, # 未预测, 湿度保持不变
        '行动': '数据缺失, 无法预测',
        '建议灌溉水量 (m³)': 0
    })
    print(f"警告: {current_date.date()} 的部分气象数据缺失。跳过当天预测。")
    continue

# --- 1. 计算 Penman-Monteith ET0 (mm/day) ---
# 大气压 (kPa) - FAO-56 Eq. 7
P_kPa = 101.3 * ((293 - 0.00617 * ELEVATION_M) / 293)**5.26

# 湿度常数 (kPa/°C) - FAO-56 Eq. 8
gamma = 0.0016286 * P_kPa / LAMBDA_WATER

# 饱和水汽压 (kPa) - FAO-56 Eq. 11 (使用平均温度近似)
es_avg = mean_es(temp)

# 实际水汽压 (kPa) - FAO-56 Eq. 13 (从露点温度计算)

```

```

ea_dew_point = es_calc(dew_point)

# 饱和水汽压差 (kPa)
es_minus_ea = es_avg - ea_dew_point

# 饱和水汽压曲线斜率 (kPa/°C) - FAO-56 Eq. 5
delta = (4098 * es_avg) / (temp + 237.3)**2

# 2 米高处风速 (m/s) - FAO-56 Eq. 33 (假设原始数据已经是 2 米高度)
u2 = wind

# 下行短波辐射强度 (ssrd) 从 W/m² 转换为 MJ/m²/day (日太阳辐射 Rs)
# FAO-56 Table 1: 1 W/m² = 0.0864 MJ/m²/day
Rs_MJ_m2_day = ssrd * 0.0864

# 理论晴空辐射 (Rso, MJ m⁻² day⁻¹) - FAO-56 Eq. 19
day_of_year = current_date.timetuple().tm_yday
Rso_MJ_m2_day = calculate_rso(day_of_year, LATITUDE_DEG, ELEVATION_M)

# 净短波辐射 (Rns, MJ m⁻² day⁻¹) - FAO-56 Eq. 38
Rns_MJ_m2_day = (1 - 0.23) * Rs_MJ_m2_day # 反射率 albedo = 0.23 (参考作物草地)

# 净长波辐射 (Rnl, MJ m⁻² day⁻¹) - FAO-56 Eq. 39
# Clamp Rs_MJ_m2_day / Rso_MJ_m2_day between 0.3 and 1.0 (FAO-56 Note)
if Rso_MJ_m2_day <= 0.01: # 避免除以零或极小数, 通常发生在夜晚或极阴天
    fcd = 0.3 # 假设为最低值
else:
    fcd = np.clip(Rs_MJ_m2_day / Rso_MJ_m2_day, 0.3, 1.0) # 修正光照比率

Rnl_MJ_m2_day = SIGMA * (temp + 273.15)**4 * (0.34 - 0.14 * np.sqrt(ea_dew_point)) * \
    fcd

# 净辐射 (Rn, MJ m⁻² day⁻¹) - FAO-56 Eq. 40
Rn_MJ_m2_day = Rns_MJ_m2_day - Rnl_MJ_m2_day

# Penman-Monteith ET0 公式 (mm/day) - FAO-56 Eq. 6
# 分子项
numerator = (0.408 * delta * Rn_MJ_m2_day) + (gamma * (900 / (temp + 273.15)) * u2 * es_minus_ea)
# 分母项
denominator = delta + gamma * (1 + 0.34 * u2)

# 检查分母是否接近于零, 以防除以零错误
if denominator == 0:
    et0_daily_mm = 0

```

```

else:
    et0_daily_mm = numerator / denominator

if et0_daily_mm < 0: et0_daily_mm = 0 # ET0 不能为负

# 计算作物实际蒸散发 ETc (mm/day), 使用当前阶段的 Kc
etc_daily_mm = et0_daily_mm * KC_CORN_CURRENT_DAY

# --- 2. 考虑降雨对土壤湿度的影响 ---
# 有效降雨量 (Effective Rainfall)
# 简化处理: 假设所有降雨在当天都转化为有效入渗。
effective_rain_mm = rain

# 将 mm 转换为等效的土壤湿度百分比变化 (1mm 水深在 1m 根深土壤中约等于 0.1%的体积含水率)
etc_daily_percent = (etc_daily_mm / 1000) / CORN_ROOT_DEPTH_METERS * 100
rain_daily_percent = (effective_rain_mm / 1000) / CORN_ROOT_DEPTH_METERS * 100

# --- 3. 模拟土壤湿度变化 ---
# 土壤水平衡方程: 新的预测湿度 = 上一日预测湿度 + 降雨 - 蒸散发
predicted_soil_moisture_before_irrigation = predicted_soil_moisture + rain_daily_percent -
etc_daily_percent

# --- 4. 考虑深层渗漏 (当超过田间持水量时) ---
deep_percolation_percent = 0
if predicted_soil_moisture_before_irrigation > FIELD_CAPACITY:
    deep_percolation_percent = predicted_soil_moisture_before_irrigation - FIELD_CAPACITY
    predicted_soil_moisture_before_irrigation = FIELD_CAPACITY # 湿度不能超过田间持水量

# --- 5. 做出灌溉决策 (基于预测值, 并限定日期) ---
irrigation_volume_m3 = 0
action = "无需灌溉"

current_predicted_moisture_for_decision = predicted_soil_moisture_before_irrigation

# 检查当前日期是否在允许灌溉的范围内
# 由于 simulation_start 和 simulation_end 已经限定了范围,
# 这里的 if 语句在当前设置下会始终为真, 但保留以示逻辑完整性
if IRRIGATION_START_DATE <= current_date.date() <= IRRIGATION_END_DATE:
    if current_predicted_moisture_for_decision <= OPTIMAL_MOISTURE_LOWER_BOUND:
        action = "预测触发灌溉"
        # 灌溉后目标是达到 UPPER_BOUND
        water_needed_percent = OPTIMAL_MOISTURE_UPPER_BOUND -
current_predicted_moisture_for_decision
        irrigation_volume_m3 = (water_needed_percent / 100.0) * FIELD_AREA_SQ_METERS *

```


CORN_ROOT_DEPTH_METERS

```
# 模拟灌溉后的土壤湿度（这会影响第二天的预测起始点）
predicted_soil_moisture = OPTIMAL_MOISTURE_UPPER_BOUND
else:
    # 如果不需要灌溉，则今天的预测结束湿度就是计算得出的湿度
    predicted_soil_moisture = current_predicted_moisture_for_decision
else:
    # 如果不在灌溉限定日期内，即使湿度低于下限也不灌溉（理论上，在当前模拟范围设置下，此分支不会被触发）
    action = "日期不在灌溉窗口"
    predicted_soil_moisture = current_predicted_moisture_for_decision

simulated_decisions.append({
    '日期': current_date.date(),
    '作物阶段': current_stage_name,
    '当前 Kc 值': round(KC_CORN_CURRENT_DAY, 2),
    '当日气温 (°C)': round(temp, 2),
    '当日露点温度 (°C)': round(dew_point, 2),
    '当日风速 (m/s)': round(wind, 2),
    '当日降雨量 (mm)': round(rain, 2),
    '当日辐射 (W/m²)': round(ssrd, 2),
    '预测起始湿度 (%)': round(predicted_soil_moisture_before_irrigation + etc_daily_percent -
rain_daily_percent + deep_percolation_percent, 2),
    '预测蒸散发 (mm)': round(etc_daily_mm, 2),
    '预测结束湿度 (%)': round(predicted_soil_moisture, 2),
    '行动': action,
    '建议灌溉水量 (m³)': round(irrigation_volume_m3, 2)
})

df_simulated_decisions = pd.DataFrame(simulated_decisions)
print("\n--- 基于气象数据（包含露点温度）、Penman-Monteith 模型和作物阶段 Kc 的模拟灌溉决策 ---")
print(f"模拟灌溉日期范围：从 {simulation_start.date()} 到 {simulation_end.date()}")

# 移除了打印每日详细表格的代码行：
# print(df_simulated_decisions.set_index('日期'))

# === 新增：输出每次灌溉的日期和灌水量 ===
print("\n--- 详细灌溉事件列表 ---")
irrigation_events = df_simulated_decisions[df_simulated_decisions['行动'] == '预测触发灌溉']
if not irrigation_events.empty:
    for index, row in irrigation_events.iterrows():
        print(f"日期：{row['日期']} | 灌溉水量：{row['建议灌溉水量 (m³)']} m³")
else:
```

```

    print("模拟期间未触发任何灌溉事件。")
# =====

# 总结模拟结果
total_sim_irrigations = df_simulated_decisions[df_simulated_decisions['行动'] == '预测触发灌溉']
                        .shape[0]
total_water_used_m3 = df_simulated_decisions[df_simulated_decisions['行动'] == '预测触发灌溉']['
建议灌溉水量 (m³)'].sum()

print(f"\n 模拟总灌溉次数: {total_sim_irrigations}")
print(f"模拟总用水量: {total_water_used_m3:.2f} m³")

# ===== 可视化展示灌水土壤湿度的变化 =====
plt.figure(figsize=(15, 7))
plt.plot(df_simulated_decisions['日期'], df_simulated_decisions['预测结束湿度 (%)'],
         marker='o', linestyle='-', color='skyblue', label='预测结束土壤湿度')

# 添加最适湿度区间
plt.axhline(y=OPTIMAL_MOISTURE_LOWER_BOUND, color='red', linestyle='--', label=f'最适湿度下限
({OPTIMAL_MOISTURE_LOWER_BOUND}%)')
plt.axhline(y=OPTIMAL_MOISTURE_UPPER_BOUND, color='green', linestyle='--', label=f'最适湿度上限
({OPTIMAL_MOISTURE_UPPER_BOUND}%)')

# 标记灌溉事件 (调整了 s 参数, 使其更大更明显)
irrigation_dates = df_simulated_decisions[df_simulated_decisions['行动'] == '预测触发灌溉']['日期']
irrigation_moisture = df_simulated_decisions[df_simulated_decisions['行动'] == '预测触发灌溉']['
预测结束湿度 (%)']
if not irrigation_dates.empty:
    # 确保只在图例中显示一次“灌溉事件”
    plt.scatter(irrigation_dates, irrigation_moisture, color='purple', marker='^', s=150,
zorder=5, label='灌溉事件')
    # 绘制垂直线以更清晰地指示灌溉日期, 不添加额外的图例项
    for date in irrigation_dates:
        plt.axvline(x=date, color='purple', linestyle=':', linewidth=1.5, alpha=0.7)

# 自定义图表
plt.title(f'2025 年玉米 ({simulation_start.date()} - {simulation_end.date()}) 土壤湿度变化与灌溉决策
\n(作物面积: {FIELD_AREA_SQ_METERS} m²)\n 注意: 图中灌溉事件点可能因日期密集而重叠, 实际触发次数请参考下
方文字输出')
plt.xlabel('日期')
plt.ylabel('土壤湿度 (%)')
plt.ylim(min(OPTIMAL_MOISTURE_LOWER_BOUND - 5, df_simulated_decisions['预测结束湿度 (%)'].min()) -

```

```
5),  
        max(OPTIMAL_MOISTURE_UPPER_BOUND + 5, df_simulated_decisions['预测结束湿度 (%)'].max() +  
5))  
plt.grid(True, linestyle='--', alpha=0.7)  
plt.legend()  
plt.xticks(rotation=45, ha='right')  
plt.tight_layout()  
plt.show()
```